

Technical Document – Lab 2

1 – Input System

The Input System package was installed through the Package Manager, and an Input Actions asset was created inside a folder named "Actions".

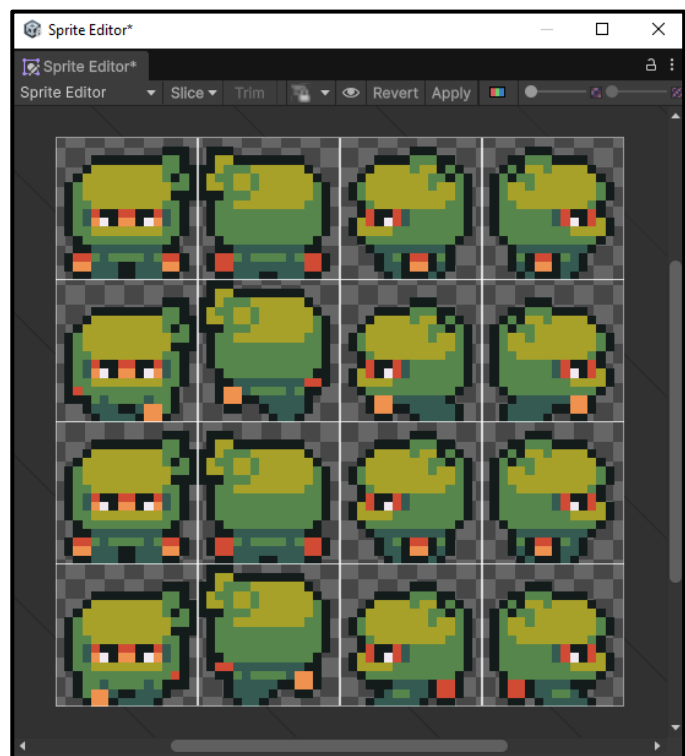
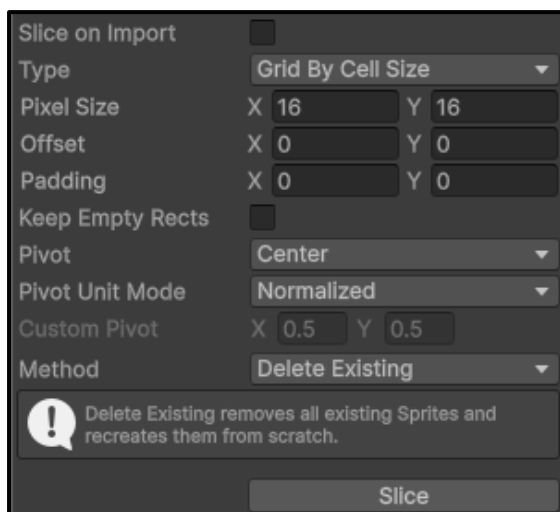
Inside the asset, a "Movement" Action Map and a "Move" action were created. The action type was set to "Value" with a "Vector2" control type, and an Up/Down/Left/Right Composite was added and assigned to the keyboard keys W, S, A, D, respectively. After saving changes, a C# file with the same name was generated.

2 – Sprite Preparation

The Green Ninja sprites were selected, and their Pixels Per Unit was set to 16, Filter Mode to Point (no filter) and Compression to None. The changes were applied, with the Sprite Mode from individual sprites set to single and spritesheets set to multiple.

The Walk spritesheet was opened in the Sprite Editor. The slicing mode was set to Grid by Cell Size, with both the X and Y pixel sizes set to 16. This separated the spritesheet into individual sprites that could be used in animation clips.

The sprite "Walk_0" was selected and renamed "PlayerNinja". A new sorting layer called Player was created in the Sprite Renderer settings and assigned to the character.



3 – Movement Script

Inside Assets, the folders Scripts and Player were created. The PlayerMovement.cs script was created inside Scripts/Player and added to PlayerNinja.

In PlayerMovement.cs, a speed variable was created, with a "Config" header placed above it. A variable to access the Input Actions asset was also created and initialized in Awake(). The OnEnable() and OnDisable() methods were added to enable and disable Input Actions when necessary.

A Rigidbody2D component was added to PlayerNinja, with Gravity Scale set to zero and rotation on the Z axis frozen. The Rigidbody2D variable was added to the script and initialized using "GetComponent<Rigidbody2D>()".

The moveDirection variable and the ReadMovement(), Move(), FixedUpdate() and Update() methods were added. The character's speed was adjusted in the Inspector, and the movement was tested in play mode.

4 – Walking Animations

With PlayerNinja selected, the Animation tab was opened. An Animations folder and a Player subfolder were created. The first animation, Walk_Down, was created and saved inside this folder.

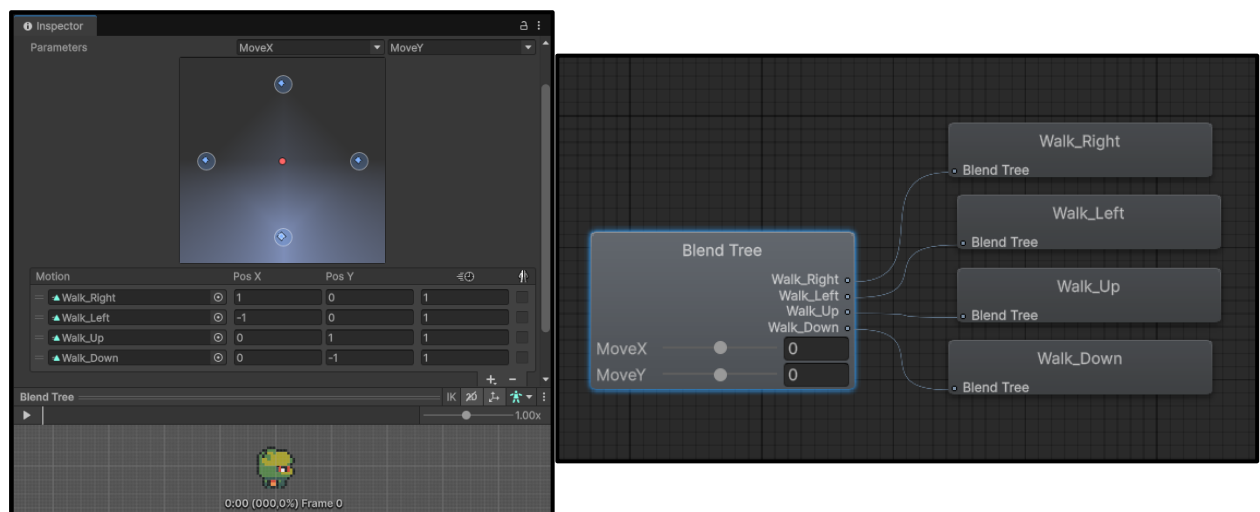
Walk_0 was placed at the beginning of the timeline, Walk_4 at frame 10, Walk_8 at frame 20 and Walk_12 at frame 30. The first keyframe was copied and pasted at frame 40 to complete the animation cycle.

The same process was repeated to create Walk_Up, Walk_Left and Walk_Right, using the corresponding sprites for each direction.

5 – Blend Tree Setup

The automatically created animation nodes were deleted from the Animator, and a Blend Tree was created. The float parameters "MoveX" and "MoveY" were added, and the Blend Type was set to 2D Simple Directional.

The four walking animations were assigned to separate Motions. Their directional values were configured as right (1, 0), left (-1, 0), up (0, 1) and down (0, -1). The MoveX and MoveY values in the main Blend Tree node were set to zero.



6 – Animations => Script

A private Animator variable was added to PlayerMovement.cs and initialized in Awake() using "GetComponent<Animator>()". The ReadMovement() method was updated with a condition to handle the absence of movement input.

The MoveX and MoveY Animator parameters were connected to the movement direction through "animator.SetFloat()". The character's movement and animations were then tested together.

7 – Idle Animations (Bonus)

The Idle spritesheet was sliced using the same settings as the Walk spritesheet. Four animations were created: Idle_Down, Idle_Up, Idle_Left and Idle_Right, and saved in the Animations/Player folder.

A Blend Tree called Idle Tree was created, and the four idle animations were assigned to separate Motions with their corresponding directional values.

A boolean parameter called Moving was added to the Animator, and Entry was connected to Idle Tree. Transitions were created between Idle Tree and the walking Blend Tree. Exit Time and Fixed Duration were disabled, and Transition Duration was set to zero. The conditions Moving = true and Moving = false were assigned to the respective transitions.

Finally, the PlayerMovement.cs script was updated to control the Moving parameter according to the movement input, allowing the character to switch automatically between idle and walking animations.